01 / RETAILOPS

Mission et architecture

Tchoupou Pindoh Jordan
M1 Data & IA — NEXA
Rapport technique • 2 novembre 2026

Contexte

RetailOps est une entreprise fictive de commerce électronique. Le projet transforme quatre fichiers CSV synthétiques en une base PostgreSQL locale, sécurisée et exploitable. Les objectifs sont la reproductibilité du déploiement, le contrôle des accès, la traçabilité et la récupération des données.

Environnement retenu

L'environnement testé sur le Mac Intel utilise PostgreSQL 17.11 Alpine et pgAudit 17.1. Une image personnalisée compile pgAudit dans une étape de construction séparée ; seuls les fichiers de l'extension sont copiés dans l'image finale. Alpine a été retenu après une erreur d'exécution rencontrée avec la variante Debian dans cet environnement.

Organisation du déploiement

Docker Compose expose le service db uniquement sur 127.0.0.1, port 5433 par défaut. PostgreSQL écoute dans le conteneur sur le port 5432. Le volume nommé pgdata contient le cluster ; data/ et sql/ sont montés en lecture seule. backups/ reçoit les sauvegardes et les archives WAL. Un redémarrage ne supprime pas le volume.

Choix de PostgreSQL

Les relations entre clients, commandes et produits, les contraintes d'intégrité et les transactions justifient une base relationnelle. PostgreSQL fournit aussi TLS, les rôles, les plans SQL et les vues d'exploitation nécessaires. Un stockage documentaire ne présente pas ici de besoin établi. Ce laboratoire ne prétend pas démontrer une architecture distribuée ou de production.

Preuve de persistance

Les nombres de lignes ont été relevés avant et après redémarrage du conteneur. La comparaison n'a signalé aucune différence pour les quatre tables. Cette vérification porte sur les volumes ; elle ne constitue pas une comparaison de chaque valeur. Preuves : evidence/persistance.txt et persistance_avant/apres.txt.

02 / RETAILOPS

Modèle relationnel et qualité

Relations

customers (1) est reliée à orders (N) par customer_id. orders (1) est reliée à order_items (N) par order_id. products (1) est reliée à order_items (N) par product_id. Chaque table possède une clé primaire entière avec génération d'identifiant par défaut.

Contraintes

Les clés étrangères empêchent les références inexistantes. NOT NULL impose les champs requis ; les emails et SKU sont uniques. Les prix sont numériques et positifs ou nuls ; les quantités sont strictement positives. Les statuts acceptés sont pending, paid, cancelled et refunded ; les canaux sont web, mobile et store.

Import transactionnel

Les CSV sont chargés avec en-tête et encodage UTF-8, dans l'ordre clients, produits, commandes puis lignes de commande. L'import est encadré par BEGIN et COMMIT. Les séquences sont recalées sur les identifiants importés pour éviter les collisions lors des insertions futures. Aucun retrait arbitraire de ligne n'est effectué.

Volumes et contrôles

300 clients, 80 produits, 5 000 commandes et 9 989 lignes de commande sont importés. Les sept contrôles SQL retournent zéro anomalie : absence de doublons d'identifiants et absence de références invalides pour les trois relations. Les contraintes complètent ces contrôles au moment du chargement. Preuves : import.txt et controles.txt.

Trois requêtes métier

Le script 04 calcule les montants des commandes payées par mois, les dix produits les plus vendus et les montants/paniers moyens par région. Les commandes annulées, en attente et remboursées sont exclues par le filtre paid. Le panier moyen est calculé après agrégation par commande afin de ne pas donner davantage de poids aux commandes comportant plus de lignes.

Résultats illustratifs

L'Auvergne-Rhône-Alpes totalise 597 647,24 en montants payés et 722 commandes. L'Occitanie affiche le panier moyen le plus élevé, 857,24. Aucune devise n'est imposée par les colonnes fournies ; ces valeurs sont présentées comme montants du jeu synthétique. Source : evidence/requetes_metier.txt.

03 / RETAILOPS

Sécurité, audit et données

Matrice des accès

retail_ro et retail_rw sont des groupes NOLOGIN. analyst_login hérite de retail_ro : lecture des quatre tables. app_login hérite de retail_rw, lui-même membre de retail_ro : lecture, insertion dans orders/order_items et modification de la seule colonne status de orders. Les séquences nécessaires aux insertions sont autorisées. Ces comptes ne sont pas superutilisateurs.

Authentification et transport

Les connexions réseau utilisent SCRAM-SHA-256. pg_hba.conf refuse les connexions hostnossl et autorise les deux comptes métier en hostssl sur retail_admin. L'administration par socket local dans le conteneur conserve une règle trust pour postgres : son périmètre doit rester réservé à l'administrateur Docker.

TLS vérifié

Une connexion analyst_login avec sslmode=verify-full et le certificat local explicitement approuvé a réussi ; pg_stat_ssl indique TLSv1.3 et ssl=t. Une connexion sslmode=disable est refusée par pg_hba.conf. Le certificat autosigné CN=localhost convient à l'essai local ; une production nécessite une gestion de confiance, de renouvellement et de noms adaptée. Preuves : tls_verified.txt, tls_refus.txt et reproduction_tls.txt.

Preuves des permissions

Une connexion réelle de l'analyste a permis de lire 5 000 commandes. Son UPDATE a été refusé. L'UPDATE du statut par app_login a réussi puis a été annulé. Le script 06 rejoue les autorisations via SET ROLE : il vérifie les permissions, pas l'authentification par mot de passe. Les preuves initiales et de reconstruction sont conservées.

Audit proportionné

pgAudit est préchargé, créé comme extension et configuré pour write, ddl, role. Une trace AUDIT de classe WRITE et commande UPDATE a été relevée, y compris pour une écriture annulée. Le texte SQL et les paramètres sont désactivés dans ces événements. Les journaux techniques restent à examiner avant diffusion.

RGPD : limites explicites

Le jeu est synthétique. La note docs/rgpd.md décrit finalités, accès, conservation et risques. L'analyste peut encore lire les noms et emails : des vues minimisées seraient nécessaires avec des données réelles. Les durées proposées ne sont pas appliquées par une purge automatique. Les dumps ne sont pas chiffrés par le projet.

Transactions et exploitation

COMMIT et ROLLBACK

Le script 14 utilise une table temporaire : un solde initial de 100 passe à 80 puis est validé par COMMIT. Une seconde opération le fait passer temporairement à 50 ; ROLLBACK rétablit 80. Les deux indicateurs de validation valent t. La table temporaire est supprimée et les données métier sont préservées. Preuve : evidence/transactions.txt.

Blocage observé

Deux sessions ont modifié la commande 1 dans des transactions ouvertes. B (PID 1704) attendait A (PID 1283) : wait_event_type=Lock, wait_event=transactionid et pg_blocking_pids(1704)={1283}. Les identifiants sont ceux de cet essai et ne sont pas constants.

Diagnostic et résolution

pg_locks montrait un ExclusiveLock accordé à A sur la transaction 798 et un ShareLock non accordé à B sur la même transaction. ROLLBACK dans A a libéré le verrou ; B a terminé son UPDATE puis a également exécuté ROLLBACK. Il s'agissait d'une attente simple et non d'un deadlock. docs/verrous.md conserve les observations transcrites et la procédure, pas une capture brute indépendante du test.

Espace et maintenance

La supervision relève une base de 9 326 kB. Après actualisation des statistiques, les estimations de lignes sont cohérentes avec les volumes importés et n_dead_tup vaut zéro. VACUUM ANALYZE a été exécuté sur orders. VACUUM rend l'espace des anciennes versions réutilisable ; il ne garantit pas une diminution de la taille du fichier sur disque.

Tablespaces

Le cluster conserve pg_default et pg_global. Un tablespace personnalisé n'apporte pas de bénéfice mesuré sur ce petit déploiement local mono-machine et rendrait la portabilité plus complexe. Une séparation sur des stockages distincts pourrait être étudiée en présence d'un besoin réel.

Prévention

Terminer systématiquement les transactions et réduire leur durée limite les blocages. pg_stat_activity, pg_locks et pg_blocking_pids permettent d'identifier la cause avant toute intervention. Un lock_timeout adapté peut borner l'attente ; il ne remplace pas le diagnostic.

Sauvegarde et continuité

Dump et restauration

Un dump custom de retail_admin a été produit, d'environ 131 Ko, puis inspecté avec pg_restore --list. Il a été restauré dans retail_restore_check avec --exit-on-error --no-owner --no-privileges. La sortie de restauration est vide, ce qui est compatible avec une exécution sans message ; les tables restaurées et leur comparaison apportent la preuve utile.

Comparaison indépendante

Les quatre volumes restaurés correspondent à ceux de la source : 300, 80, 5 000 et 9 989 lignes selon les tables. La comparaison diff est vide et le rapport indique PASS. Cette validation ne prouve pas l'égalité cellule par cellule ni une reprise des autorisations, exclues volontairement lors de cette restauration.

Archivage WAL

wal_level est replica ; archive_mode est on. archive_command copie chaque segment fermé vers /backups/wal après vérification de son absence. L'essai a créé un point de restauration puis demandé une rotation WAL. Résultat conservé : archived_count=1, failed_count=0 et un segment de 16 Mo présent. Preuve : evidence/wal.txt.

Différence avec le PITR

Une récupération à un instant cible exige une sauvegarde physique exploitable et une chaîne continue des WAL nécessaires. Le dump logique ne sert pas de sauvegarde physique de départ. Le projet démontre l'archivage, mais aucun PITR complet n'a été exécuté. Aucun délai maximal de fermeture ou d'archivage des segments n'est garanti.

RPO et RTO proposés

Objectif RPO : 24 heures avec un dump quotidien réussi et copié hors du Mac. Objectif RTO : 30 minutes pour restaurer cette petite base sur un environnement disponible et vérifier configuration, droits et données. Ces objectifs sont proposés, non mesurés ni garantis : les sauvegardes sont manuelles et aucune reprise complète chronométrée n'a été réalisée.

Risques résiduels

Les données, dumps et WAL sont sur le même ordinateur. Une perte du Mac peut affecter toutes les copies. Une exploitation durable nécessite copie externe protégée, rétention coordonnée, surveillance de l'espace, sauvegardes planifiées et essais de récupération. Le dump livré précède la création de l'index : reconstruire cet index après restauration si nécessaire.

Performance, supervision et bilan

Comparaison de plans

La même requête sélectionne les commandes du client 42 et les trie par date décroissante. Avant indexation : Seq Scan, 18 résultats, 4 982 lignes écartées, 41 shared hits au niveau racine et 0,701 ms. Après création de l'index (customer_id, ordered_at DESC) : Bitmap Index Scan puis Bitmap Heap Scan, tri conservé, 18 résultats, 20 shared hits et 0,345 ms.

Interprétation

Le temps observé diminue de 50,8 % et les shared hits au niveau racine de 51,2 %. Il s'agit de deux mesures ponctuelles sur une petite base ; cache, charge et variabilité empêchent d'en déduire un gain garanti. Les buffers des nœuds ne doivent pas être additionnés comme des valeurs indépendantes. L'index ajoute du stockage et un coût d'entretien sur les écritures.

Supervision et alertes

Le relevé conservé indique une connexion, zéro deadlock, aucun bloqueur ni requête active de plus de cinq secondes et zéro échec d'archivage. Les seuils proposés comprennent une attente de verrou supérieure à 30 secondes et toute augmentation de failed_count entre deux relevés. Des seuils complémentaires concernent les connexions, l'espace disque et les requêtes longues. La collecte et les alertes restent manuelles.

Reconstruction contrôlée

Un projet Compose distinct retailops_repro, sur le port 5434 et un volume neuf, a été initialisé par script. Il retrouve les quatre volumes, ssl=on, archive_mode=on et pgAudit 17.1. La lecture avec connexion verify-full et les autorisations SET ROLE ont été vérifiées. Les mots de passe restent configurés interactivement. L'essai a été réalisé sur le même Mac, pas sur une autre machine.

Livrables et réserves

Le dossier réunit Dockerfile, Compose, 14 scripts SQL, CSV, documentation et preuves. Le script refuse de réimporter un schéma existant et ne constitue pas un outil de migration. La branche pgAudit reste mutable. Les preuves initiales sont conservées sans les présenter comme une nouvelle exécution par le rédacteur du rapport. Les outils Docker/PostgreSQL ne sont pas disponibles dans l'environnement de revue : aucune relance indépendante du service n'y a été effectuée.

Références et suite

Sources : énoncé RetailOps fourni, fichiers du projet et evidence/. Références techniques : postgresql.org/docs/17/, github.com/pgaudit/pgaudit et docs.docker.com/compose/. Priorités suivantes : figer la révision pgAudit, renforcer les tests automatiques avec verdicts, limiter les données visibles et tester une reprise complète. La publication GitHub exclura secrets, clés privées, dumps et WAL.